

CampusConnect — Complete Technical Architecture & Codebase Guide

Full-Stack MERN Architecture, Database Schemas, Environment Variables, File-by-File Code Walkthrough & Data Flows

Project Platform: CampusConnect MERN Stack	Database: MongoDB Atlas Cloud Cluster
Frontend: React 18, Vite 5, Tailwind CSS	Backend API: Node.js, Express 4, Mongoose 8
Authentication: JWT Bearer Token + Bcrypt	Document Type: Official Technical Master Guide

1. Technology Stack & Package Inventory

CampusConnect is built using the MERN Stack. Every single package, library, and framework used in the codebase is listed below with its exact role:

Layer	Technology / Package	Version	Role & Technical Implementation Details
Frontend UI	React.js	v18.2.0	Component-based SPA library rendering dynamic user interface views.
Build Tool	Vite	v5.2.0	Lightning-fast frontend build tool and development server with instant HMR.
Styling	Tailwind CSS	v3.4.3	Utility-first CSS framework providing responsive layouts and dark accents.
Routing	React Router DOM	v6.22.3	Client routing with role-based `ProtectedRoute` guards for Admin, Faculty & Student.
HTTP Client	Axios	v1.6.8	Promise-based HTTP client pre-configured with interceptors inserting Bearer JWT.
Analytics	Recharts	v3.10.1	Chart library rendering SVG BarCharts and PieCharts for department metrics.
Icons	Lucide React	v0.368.0	Modern SVG icon set used across headers, sidebars, cards, and buttons.
PDF Export	jsPDF & autoTable	v4.2.1	Client-side PDF report generator allowing students to download verified records.
Backend	Node.js	v18/v20	Event-driven asynchronous JavaScript server-side runtime.
Web Server	Express.js	v4.19.2	Web application framework handling RESTful routing, middleware, and errors.
Database	MongoDB Atlas	Cloud	Distributed NoSQL document database storing users, classes, subjects & attendance.
ODM	Mongoose	v8.3.1	Object Data Modeling library providing schemas, relationships, and queries.
Auth Tokens	JSON Web Token (JWT)	v9.0.2	Stateless session authentication issuing signed JWT tokens containing `id` & `role`.
Password Hashing	Bcrypt.js	v2.4.3	Salted password hashing algorithm (10 rounds) preventing plain-text password storage.

2. Environment Variables, Secrets & Security Configuration

Sensitive parameters are isolated into environment variables (`.env`). These are loaded via `dotenv.config()` in Node.js and `import.meta.env` in Vite:

Variable Name	Layer	Example / Value	Why Used & Security Rules
PORT	Backend	5000	Specifies Express server port. Injected dynamically by Render/Vercel.
MONGODB_URI / MONGO_URI	Backend	mongodb+srv://user:pass@cluster.mongodb.net/campusconnect	Atlas URI containing db credentials & cluster host. Supported simultaneously in `config/db.js`.
JWT_SECRET	Backend	campusconnect_jwt_secret_2026	Private key used by `jsonwebtoken` to sign JWT tokens. Prevents token forgery.
NODE_ENV	Backend	production / development	Controls error stack verbosity. Suppresses sensitive logs in production.

VITE_API_URL	Frontend	https://backend.onrender.com/api	Base URL for Axios requests. Switches between localhost and live backend API.
--------------	----------	----------------------------------	---

Security Rule: `.env` files are added to `.gitignore` so private database passwords and secret keys are NEVER committed to GitHub.

3. Database Architecture & MongoDB Schemas (`server/models/`)

All MongoDB collections, schema fields, and relationships defined in Mongoose:

- User.js (`User` Model): Schema Fields:** `name`, `email` (unique, indexed), `password` (bcrypt hashed), `role` (`admin`, `faculty`, `student`), `department` (String), `profileInfo` (`rollNumber`, `employeeId`, `phone`, `bio`, `avatar`).
Hooks & Methods: Pre-save hook hashes password using `bcrypt.hash(pass, 10)`. Instance method `matchPassword()` compares candidate password.
- Department.js (`Department` Model): Schema Fields:** `name` (e.g. `Information Technology`), `code` (e.g. `IT`), `description`.
Role: Stores academic departments. Serves as parent reference for classes and faculty.
- AcademicClass.js (`AcademicClass` Model): Schema Fields:** `name` (e.g. `IT-D1`), `department` (Ref `$>` `Department`), `year` (e.g. `3rd Year`), `semester` (e.g. `Semester 5`).
Role: Represents class divisions. Groups students for attendance and subject allocation.
- StudentEnrollment.js (`StudentEnrollment` Model): Schema Fields:** `student` (Ref `$>` `User`), `academicClass` (Ref `$>` `AcademicClass`), `department` (Ref `$>` `Department`), `rollNumber` (`01` to `70`), `status` (`active`).
Role: Links student accounts to their assigned class division and roll number.
- Subject.js (`Subject` Model): Schema Fields:** `name` (e.g. `Web Engineering`), `code` (e.g. `IT303`), `faculty` (Ref `$>` `User`), `academicClass` (Ref `$>` `AcademicClass`), `department` (Ref `$>` `Department`).
Rule: Created ONCE by faculty and permanently retained under 'My Subjects'.
- AttendanceSession.js (`AttendanceSession` Model): Schema Fields:** `subject` (Ref `$>` `Subject`), `academicClass` (Ref `$>` `AcademicClass`), `faculty` (Ref `$>` `User`), `date` (`YYYY-MM-DD`), `sessionTime` (e.g. `10:00 AM`).
Role: Recorded date-by-date when faculty submits attendance for a subject session.
- AttendanceRecord.js (`AttendanceRecord` Model): Schema Fields:** `attendanceSession` (Ref `$>` `AttendanceSession`), `student` (Ref `$>` `User`), `subject` (Ref `$>` `Subject`), `academicClass` (Ref `$>` `AcademicClass`), `status` (`Present` / `Absent`).
Role: Individual student status per session. Queried to calculate attendance percentage.
- Notification.js (`Notification` Model): Schema Fields:** `recipient` (Ref `$>` `User`), `title`, `message`, `type` (`attendance`, `event`, etc.), `isRead` (Boolean, default `false`).
Role: Stores student alerts. Created automatically when attendance is recorded or updated.

4. Backend Codebase & File Breakdown (`server/`)

Detailed file breakdown of backend controllers, middleware, and routes:

File Path	Module Role	Responsibilities, Functions & Data Operations Executed
<code>server.js</code>	Entry Point	Configures Express app, CORS headers, JSON middleware, initializes <code>connectDB()</code> , mounts routes (<code>/api/auth</code> , <code>/api/attendance</code> , etc.), and listens on <code>PORT</code> .
<code>config/db.js</code>	DB Connection	Mongoose MongoDB Atlas connection. Includes Google DNS SRV lookup fallback (<code>8.8.8.8</code>) for Windows lookup safety and local MongoDB fallback.
<code>middleware/authMiddleware.js</code>	Security Guards	Exports <code>protect</code> (verifies JWT token from <code>Authorization: Bearer</code> header) and <code>authorize(...roles)</code> enforcing role-based access control.
<code>middleware/errorMiddleware.js</code>	Error Handler	Centralized Express error handling middleware formatting exceptions into structured JSON error responses <code>{ message, stack }</code> .
<code>controllers/authController.js</code>	User Auth	<code>registerUser</code>: Validates input, hashes password, creates <code>User</code> & <code>StudentEnrollment</code>, returns JWT. <code>loginUser</code>: Verifies password via <code>matchPassword()</code>, returns JWT token. <code>getMe</code>: Returns user profile.

<code>controllers/attendanceController.js</code>	Attendance Engine	<code>`getStudentsForSession`</code>: Fetches class roster; checks if session exists for <code>`date`</code> (enters Edit Mode if found). <code>`recordAttendanceSession`</code>: Saves session & records, executes <code>`Notification.create()`</code> for all students. <code>`updateAttendanceSession`</code>: Updates status & dispatches update notifications. <code>`getStudentAttendanceSummary`</code>: Calculates subject & overall percentage with 0-lecture safety.
<code>controllers/academicController.js</code>	Academic Specs	<code>`createSubject`</code>: Creates permanent <code>`Subject`</code> document. <code>`getMySubjects`</code>: Returns subjects owned by logged-in faculty. <code>`getSubjectsForClass`</code>: Returns subjects for a class division.
<code>controllers/departmentController.js</code>	Department Admin	Handles <code>`getDepartments`</code> , <code>`createDepartment`</code> , and <code>`deleteDepartment`</code> API endpoints for administrators.
<code>controllers/userController.js</code>	User Admin	Provides <code>`getAllUsers`</code> , <code>`getFacultyMembers`</code> , <code>`getStudentsByClass`</code> , and user deletion methods.
<code>controllers/notificationController.js</code>	Notifications	Provides <code>`getNotifications`</code> (fetches top 30 user notifications), <code>`markAsRead`</code> , and <code>`markAllAsRead`</code> .
<code>routes/*.js</code>	REST API Routes	Maps HTTP endpoints (<code>`GET`</code> , <code>`POST`</code> , <code>`PUT`</code> , <code>`DELETE`</code>) to corresponding controller functions with <code>`protect`</code> & <code>`authorize`</code> guards.

5. Frontend Codebase & UI Component Breakdown (``client/``)

Detailed file breakdown of client-side React pages, services, and components:

Component / Page Path	Layer / Type	UI Workflow, User Interaction & Internal Logic Executed
<code>src/main.jsx & App.jsx</code>	App Bootstrap	Initializes React DOM root, mounts <code>`BrowserRouter`</code> , and configures top-level layout wrappers.
<code>src/routes/AppRoutes.jsx</code>	Client Router	Defines all client routes (<code>`/login`</code> , <code>`/register`</code> , <code>`/student/attendance`</code> , <code>`/faculty/subjects`</code> , <code>`/analytics`</code>). Wraps routes in <code>`ProtectedRoute`</code> checking JWT role.
<code>src/services/api.js</code>	Axios Base	Creates central Axios instance with <code>`baseUrl`</code> (<code>`import.meta.env.VITE_API_URL`</code>). Attaches request interceptor inserting <code>`Authorization: Bearer `</code> from <code>`localStorage`</code> .
<code>src/components/Navbar.jsx</code>	Header Bar	Renders logo, page title, user avatar, role badge, and embeds <code>`NotificationDropdown.jsx`</code> .
<code>src/components/NotificationDropdown.jsx</code>	Bell Panel 	Renders bell icon with unread counter. Fetches <code>`/api/notifications`</code> , lists alerts with custom icons, and updates read state on click.
<code>src/components/Sidebar.jsx</code>	Navigation Drawer	Renders role-customized navigation menus (Admin sees Analytics/Departments; Faculty sees My Subjects/Attendance; Student sees Attendance/Dashboard).
<code>src/pages/faculty/TakeAttendancePage.jsx</code>	Attendance Sheet	Faculty selects date, loads student roster, detects existing attendance (Edit Mode amber banner), toggles Present/Absent per student, and submits batch attendance.
<code>src/pages/faculty/FacultySubjectsPage.jsx</code>	Subject Portal	Allows faculty to create permanent subjects for assigned classes and view all owned subjects.
<code>src/pages/student/StudentAttendancePage.jsx</code>	Student Portal	Displays overall percentage, subject breakdowns, status badges ('No Lectures', 'Good', 'Low Attendance'), and exports verified PDF reports via jsPDF.
<code>src/pages/AnalyticsPage.jsx</code>	Analytics Portal	Fetches MongoDB aggregation statistics. Renders Recharts BarChart ('Students by Department') with collision-free angled labels and PieChart ('User Role Distribution').
<code>src/pages/admin/*</code>	Admin Suite	Comprehensive administrative pages for managing Departments, Academic Classes, Subjects, Student Enrollments, Faculty Assignments, and Activity Logs.

6. Step-by-Step System Execution Flows (Data Traces)

Trace 1: Permanent Subject Creation Flow

- Logged-in Faculty opens **My Subjects** (``FacultySubjectsPage.jsx``) and submits subject creation form (e.g. ``Web Engineering``, ``IT-D1``).
- Frontend calls ``academicService.createSubject()`` $\rightarrow$ sends ``POST /api/academic/subjects`` with Bearer JWT token.

- 3. Express router passes request through `protect` & `authorize('faculty')` middleware in `authMiddleware.js`.
- 4. `createSubject` in `academicController.js` validates input, verifies faculty ObjectId, and executes `Subject.create()` in MongoDB Atlas.
- 5. Created subject is returned to frontend and permanently displayed under 'My Subjects'. Faculty never recreates it.

Trace 2: Date-Wise Attendance & Notification Dispatch Flow

- 1. Faculty selects subject `Web Engineering` and Date `2026-08-15` on `TakeAttendancePage.jsx`.
- 2. Frontend calls `attendanceService.getStudentsForSession(subjectId, classId, date)` \$rightrightarrow\$ `GET /api/attendance/session-students`.
- 3. `getStudentsForSession` in `attendanceController.js` queries `StudentEnrollment` for roster AND checks `AttendanceSession.findOne({ subject, date })`. If session exists, it returns `existingSession: true` (Edit Mode).
- 4. Faculty toggles student statuses and clicks **Submit Attendance** \$rightrightarrow\$ sends `POST /api/attendance/session`.
- 5. `recordAttendanceSession` creates `AttendanceSession` document and performs `AttendanceRecord.insertMany()`.
- 6. **Notification Trigger**: Controller iterates through roster and executes `Notification.create()` for each student with `type: 'attendance'`. Message: **Web Engineering attendance has been marked for Aug 15, 2026. Status: Present.**
- 7. When student logs in, `Navbar.jsx` / `NotificationDropdown.jsx` fetches `/api/notifications` and displays  **1** badge alert in real time!

7. Live Academic Roster & Department Summary

Current live database counts across active academic departments in MongoDB Atlas:

Department Name	Code	Faculty Members	Division 1 (D1)	Division 2 (D2)	Total Students
Information Technology	IT	3 Faculty	36 Students	35 Students	71 Students
Computer Science	CS	3 Faculty	50 Students	70 Students	120 Students
AI & Data Science	AIDS	3 Faculty	20 Students	30 Students	50 Students
Electronics & Computer Science	ECS	3 Faculty	20 Students	30 Students	50 Students
Electronics & Telecommunication	EXTC	3 Faculty	20 Students	30 Students	50 Students